

Chapitre 7

Notions objet avancées

1. Classes enveloppes

Chacun des 8 types primitifs (**boolean**, **byte**, **char**, **short**, **int**, **long**, **float** et **double**) possède une classe correspondante, appelée **classe enveloppe**, qui généralise le type.

Les 8 classes enveloppes sont **Boolean**, **Byte**, **Character**, **Short**, **Integer**, **Long**, **Float** et **Double**.

Ces classes enveloppes sont définies dans le package **java.lang** et peuvent donc être utilisées sans importation.

Les classes enveloppes permettent de représenter un type simple par un objet lorsque c'est nécessaire.

Par exemple, on peut utiliser une classe enveloppe **Integer** dans une **ArrayList** alors que cette liste n'accepte pas le type simple **int** :

```
ArrayList<int> liste = new ArrayList<int>();           // N'est pas permis
```

```
ArrayList<Integer> liste = new ArrayList<Integer>();  // Est permis
```

Les classes enveloppes fournissent aussi des constantes ou des méthodes intéressantes, comme les valeurs minimum et maximum du type, ainsi que des méthodes de conversion vers d'autres types :

<https://docs.oracle.com/en/java/javase/13/docs/api/java.base/java/lang/Integer.html>

Integer.MAX_VALUE

Constante contenant la valeur maximale d'un int ($2^{31}-1$)

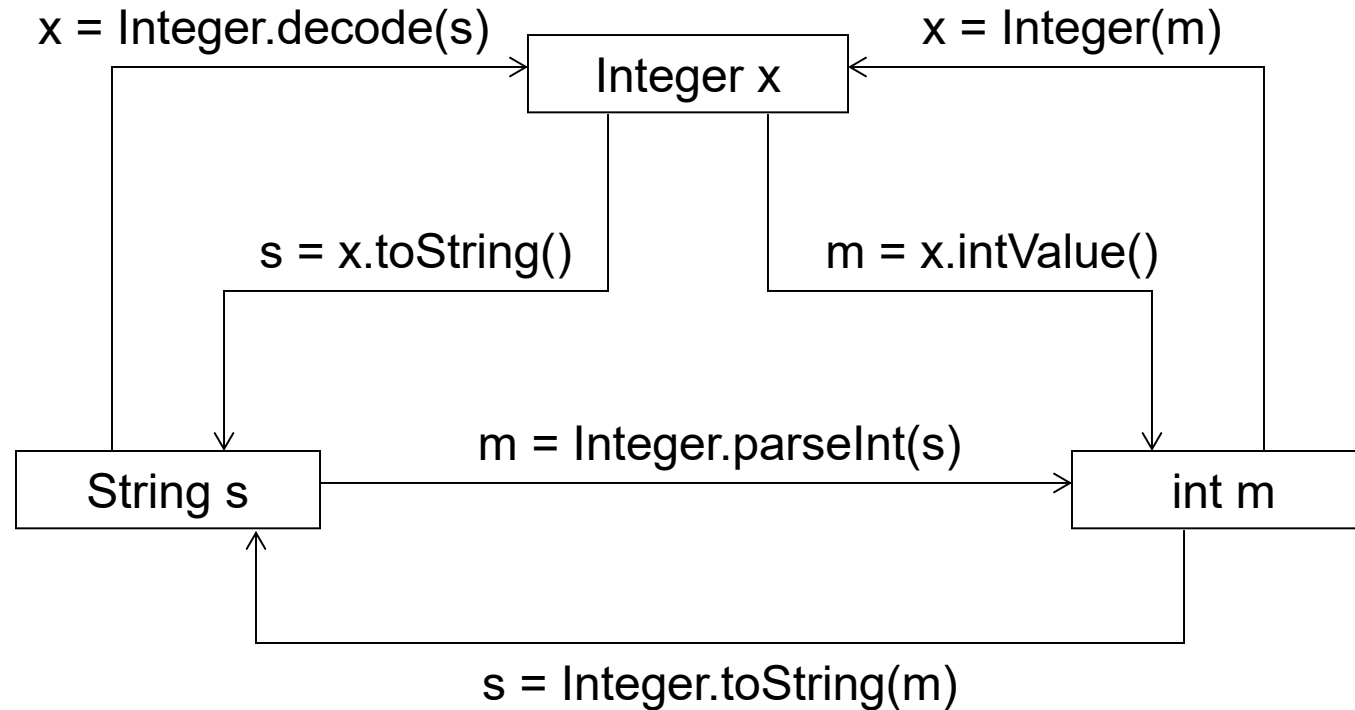
Integer.MIN_VALUE

Constante contenant la valeur minimale d'un int (-2^{31})

Integer.BYTES

Nombre d'octets utilisés pour représenter un int.

Méthodes de conversion entre le type `int` et les classes `Integer` et `String` :



Des méthodes similaires existent pour les 7 autres classes enveloppes.

2. Interfaces

En Java, une **interface** sert à définir un **contrat** que des classes doivent respecter, sans donner directement d'implémentation.

→ une interface décrit ce qu'une classe doit faire, mais pas comment elle le fait.

Les classes qui **implémentent** une interface doivent fournir le code des méthodes définies dans cette interface.

Une interface se présente comme ci-dessous : le mot-clé **interface** suivi de son nom, puis, entre accolades, une liste de **constantes** (mais pas de variable), de **prototypes** de méthodes, de **méthodes statiques**, ou de méthodes par défaut (default) qui possèdent une implémentation :

```
interface Paiement {  
    // Une constante (sera implicitement : public static final)  
    double FRAIS_TRANSACTION = 0.5;  
  
    // Méthode abstraite (contrat)  
    void payer(double montant);  
  
    // Méthode statique (accessible sans objet)  
    static void afficherConditions() {  
        System.out.println("Tous les paiements incluent des frais fixes de "  
            + FRAIS_TRANSACTION + "€.");  
    }  
  
    // Méthode par défaut (possède déjà une implémentation)  
    default void annuler() {  
        System.out.println("Le paiement a été annulé.");  
    }  
}
```

Ici, on définit une interface **Paiement** qui dit : tout moyen de paiement doit savoir exécuter un paiement.

```
interface Paiement {  
    // Une constante (sera implicitement : public static final)  
    double FRAIS_TRANSACTION = 0.5;  
  
    // Méthode abstraite (contrat)  
    void payer(double montant);  
  
    // Méthode statique (accessible sans objet)  
    static void afficherConditions() {  
        System.out.println("Tous les paiements incluent des frais fixes de "  
            + FRAIS_TRANSACTION + "€.");  
    }  
  
    // Méthode par défaut (possède déjà une implémentation)  
    default void annuler() {  
        System.out.println("Le paiement a été annulé.");  
    }  
}
```


Une interface **I** peut être **implémentée** par une classe **A**, ce qui signifie que :

- la classe **A** déclare étendre l'interface **I** ;
- toutes les méthodes dont seul le prototype figure dans l'interface **I** doivent être définies par la classe **A**.

Ici, l'interface **Paiement** comporte le prototype de la méthode **payer()**

→ Une classe qui implémente l'interface **Paiement** sera dans l'obligation de définir le corps de la méthode **payer()**.

Ensuite, différentes classes implémentent cette interface selon leur mode de fonctionnement :

CarteBancaire.java

```
class CarteBancaire implements Paiement {
    public void payer(double montant) {
        System.out.println("Paiement de " + (montant + FRAIS_TRANSACTION)
            + "€ par carte bancaire.");
    }
}
```

PayPal.java

```
class PayPal implements Paiement {
    public void payer(double montant) {
        System.out.println("Paiement de " + (montant + FRAIS_TRANSACTION)
            + "€ via PayPal.");
    }
}
```

On peut ensuite utiliser n'importe quel moyen de paiement de manière uniforme, sans se soucier de la classe exacte :

Magasin.java

```
public class Magasin {  
    public static void main(String[] args) {  
  
        // Appel de la méthode statique de l'interface  
        Paiement.afficherConditions();  
  
        Paiement paiement = new CarteBancaire();  
        paiement.payer(50.0); // Paiement de 50.5€ par carte bancaire  
        paiement.annuler();   // Appel de la méthode par défaut  
  
        paiement = new PayPal();  
        paiement.payer(75.0); // Paiement de 75.5€ via PayPal  
    }  
}
```

Si on veut ajouter un nouveau mode de paiement (ex : Google Pay), il suffit de créer une nouvelle classe qui implémente **Paiement**.

Les attributs et les méthodes d'une interface sont automatiquement publiques, ce qui entraîne qu'une classe qui implémente l'interface devra déclarer publique (avec le modificateur **public**) les méthodes de l'interface qu'elle définira.

Lorsqu'on utilise un objet d'une classe implémentant une certaine interface, on a la garantie que cet objet dispose de toutes les méthodes annoncées par l'interface.

Utilités principales :

- Définir un contrat commun : on peut traiter tous les objets qui implémentent une même interface de la même manière, même si ce sont des classes différentes.
- Polymorphisme : permet d'écrire du code générique qui fonctionne sur plusieurs types différents, tant qu'ils respectent le contrat.
- Séparation des responsabilités : l'**interface** définit *quoi faire*, la **classe** définit *comment le faire*.
- Héritage multiple : une classe peut hériter d'une seule autre classe (extends), mais peut implémenter plusieurs interfaces (implements), ce qui permet de combiner plusieurs comportements.

Remarques

Une interface :

- Ne peut pas être instanciée
- Ne contient pas de variables
- Une classe peut implémenter plusieurs interfaces
- Si une classe **A** implémente une interface **I**, les sous-classes de **A** implémentent aussi automatiquement **I**.

3. Classes abstraites

Dans une classe, une méthode est dite **abstraite** si seul son prototype figure. Elle doit alors être déclarée `abstract` :

```
abstract float surface();
```

Une classe est abstraite dès qu'elle contient une méthode abstraite, elle doit alors être déclarée `abstract` :

```
abstract class Forme
{
    abstract float surface(); // méthode abstraite
    ...
}
```

Une classe abstraite ne peut pas être instanciée.

```
Forme une_forme = new Forme() ;    // erreur
```

→ il faut **l'étendre** pour pouvoir l'utiliser. Une sous-classe d'une classe abstraite sera encore abstraite si elle ne définit pas toutes les méthodes abstraites dont elle hérite.

Exemple

Forme.java

```
abstract class Forme
{
    abstract float surface(); //méthode abstraite

    void affiche_surface()
    {
        float la_surface = surface();
        System.out.println("surface=" + la_surface);
    }
}
```

Rectangle.java

```
class Rectangle extends Forme
{
    private float longueur, largeur;

    Rectangle(float lon, float lar)
    {
        longueur = lon;
        largeur = lar;
    }

    float surface()
    {
        return longueur*largeur;
    }
}
```

EssaiFormes.java

```
class EssaiFormes
{
    public static void main(String[] argv)
    {
        Rectangle rect = new Rectangle(4,3);
        System.out.println(rect.surface());
    }
}
```

Quand utiliser quoi ?

Interface :

- Quand on veut définir un contrat sans implémentation lourde
- Quand plusieurs classes sans lien direct doivent partager ce contrat.

→ **interface = contrat**

Classe abstraite :

- Quand on veut factoriser du code commun ET imposer des méthodes à implémenter.
- Quand les classes ont une relation hiérarchique claire ("est un(e)").

→ **Classe abstraite = squelette d'implémentation**